Report: Assigment 2

Andrei Zhdanov

1 Methodology

1.1 System Architecture

The simple search engine contains 3 steps:

1. **Data Preparation (PySpark)** — reads raw documents, cleans text, and upload input to HDFS.
2. **Indexing (Hadoop MapReduce + Cassandra)** — three MapReduce pipelines compute term frequencies, document frequencies, and corpus statistics; results are stored in Cassandra.
3. **Ranking (PySpark + Cassandra)** — retrieves index data from Cassandra, computes BM25 scores via Spark, and returns the top-10 most relevant documents.

1.2 Data Preparation

`prepare_data.py` uses PySpark to:

- Read 1 000 Wikipedia articles stored as individual `.txt` files in `data/`.
- Build a single tab-separated file in HDFS (`/input/data`) with the format `<doc_id>\t<doc_title>\t<doc_text>`.
- Preprocessing: tabs and newlines are replaced with spaces; filenames are sanitized.

1.3 Indexing Pipeline (MapReduce)

1.3.1 Pipeline 1 — Term Frequency (TF)

- **Mapper** (`mapper1.py`): reads each document line, tokenizes the text (lowercase, remove punctuation, split by whitespace), and emits `<term>\t<doc_id>\t<title>\t1\t<dl>` for each token.
- **Reducer** (`reducer1.py`): aggregates counts per (term, doc_id) pair → `<term>\t<doc_id>\t<title>\t<tf>\t<dl>`.
- Result stored in `/indexer/index`.

1.3.2 Pipeline 2 — Document Frequency (DF)

- **Mapper** (`mapper2.py`): reads Pipeline 1 output, emits `<term>\t1`.
- **Reducer** (`reducer2.py`): aggregates document counts per term → `<term>\t<df>`.
- Result stored in `/indexer/vocab`.

1.3.3 Pipeline 3 — Document & Corpus Statistics

- **Mapper** (`mapper3.py`): reads raw input, tokenizes, emits `DOC_STAT\t<doc_id>\t<title>\t<dl>`.
- **Reducer** (`reducer3.py`): collects all docs, outputs per-doc stats and a summary line `CORPUS_STATS\t<N>\t<avg_dl>\t<total>`.
- Result stored in `/indexer/stats`.

1.4 Cassandra Storage Schema

Keyspace: `search_engine`. Four tables:

- **term_index** — primary key `(term, doc_id)`, columns: `doc_title`, `tf`, `dl`.
- **vocabulary** — primary key `term`, column: `df`.
- **doc_stats** — primary key `doc_id`, columns: `doc_title`, `dl`.
- **corpus_stats** — primary key `id`, columns: `num_docs`, `avg_dl`, `total_tokens`.

Batch inserts (batch size 50) are used for throughput.

1.5 BM25 Ranking

The ranking engine (`query.py`) implements BM25:

$$BM25(q,d) = \sum_{t \in q} \log\left(\frac{N}{df(t)}\right) \cdot \frac{(k_1+1)\cdot tf(t,d)}{k_1 \cdot \left[(1-b) + b \cdot \frac{dl(d)}{avg_dl}\right] + tf(t,d)} \tag{1}$$

Parameters: $k_1 = 1.2$, $b = 0.75$.
Steps:

1. Tokenize query (same preprocessing as indexer).
2. Retrieve index data from Cassandra.
3. Create a PySpark RDD from index entries.
4. `map()` — compute per-term BM25 contribution.
5. `reduceByKey()` — aggregate scores per document.
6. Sort descending, take top 10.

2 How to Run

2.1 Prerequisites

- Docker & Docker Compose installed.
- 1,000 document files in `app/data/`.

2.2 Launch

```
docker compose up
```

This will:

1. Start Hadoop and Cassandra containers.
2. Install dependencies.
3. Prepare data in HDFS.
4. Run indexing pipelines.
5. Store index in Cassandra.
6. Execute sample queries.

2.3 screenshots

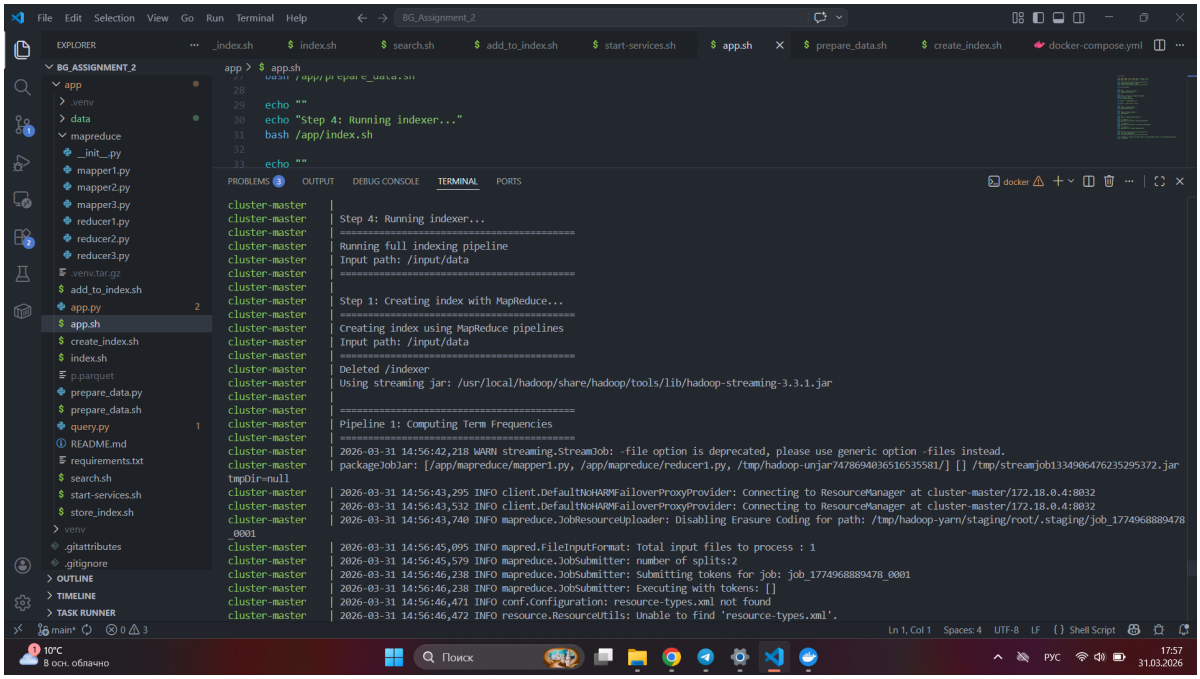

Figure 1:

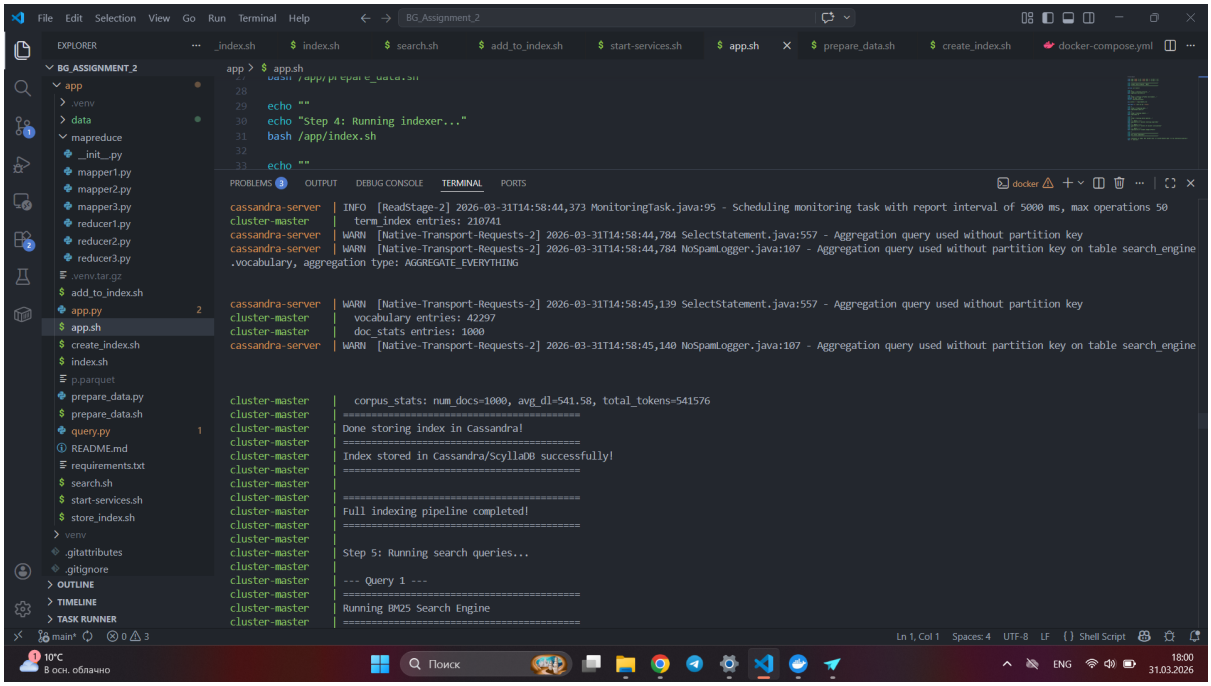

Figure 2:

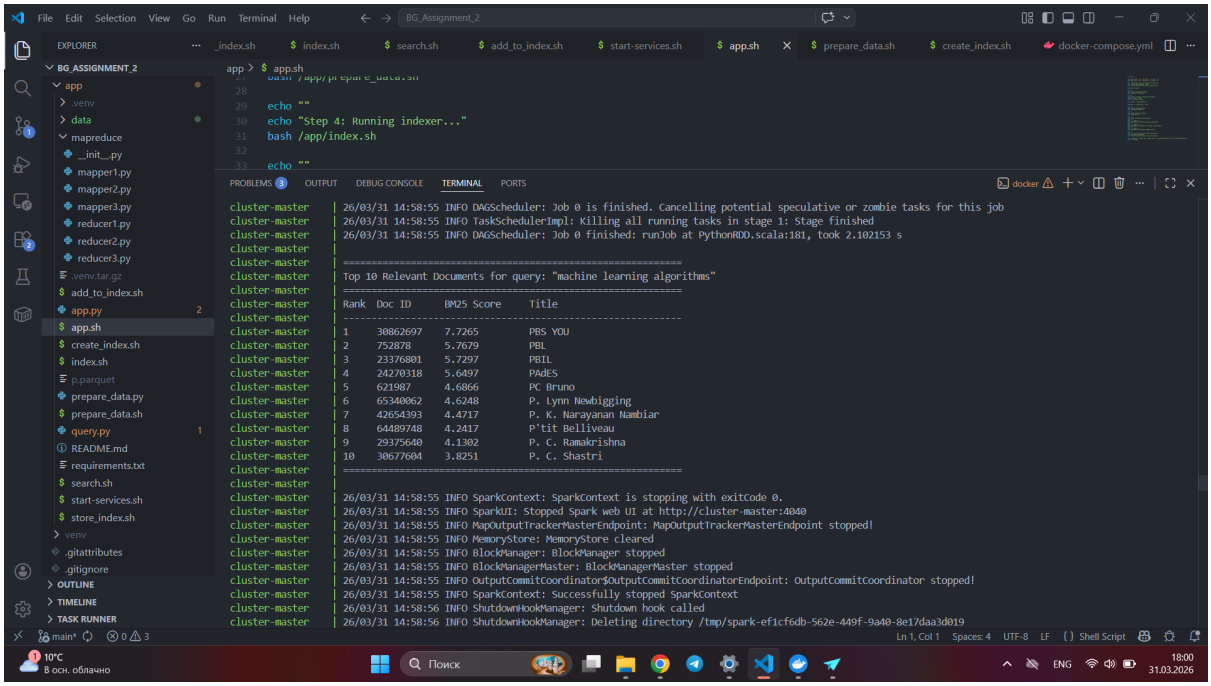

Figure 3:

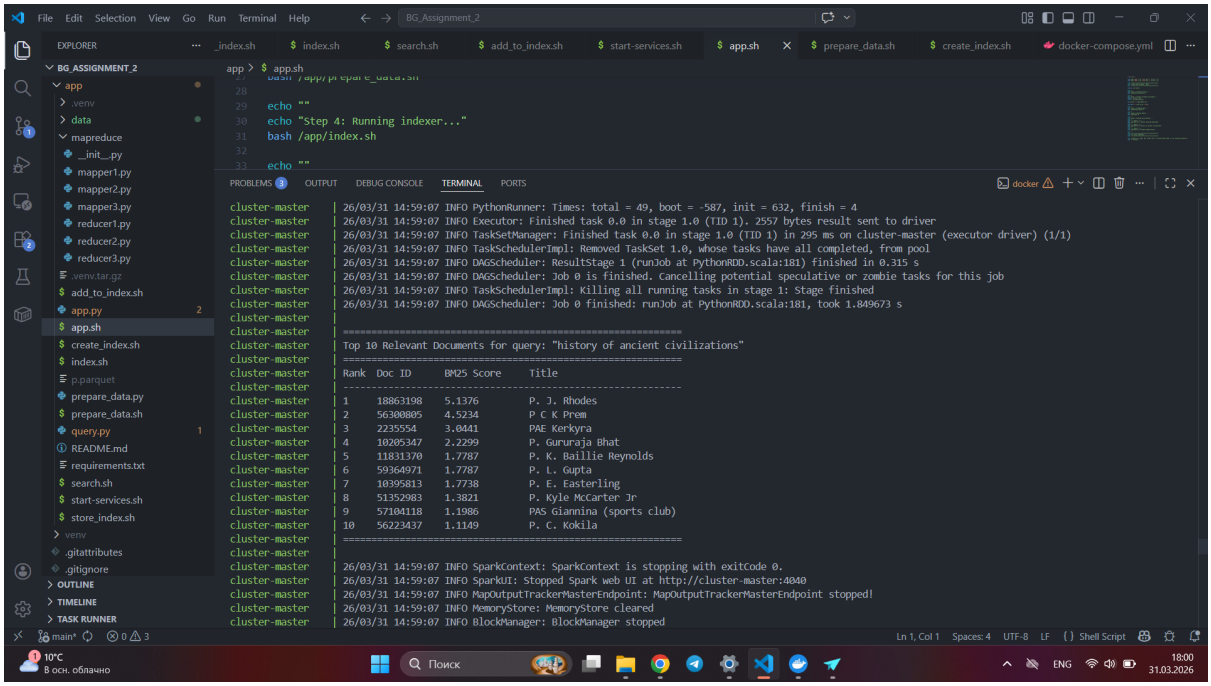

Figure 4:

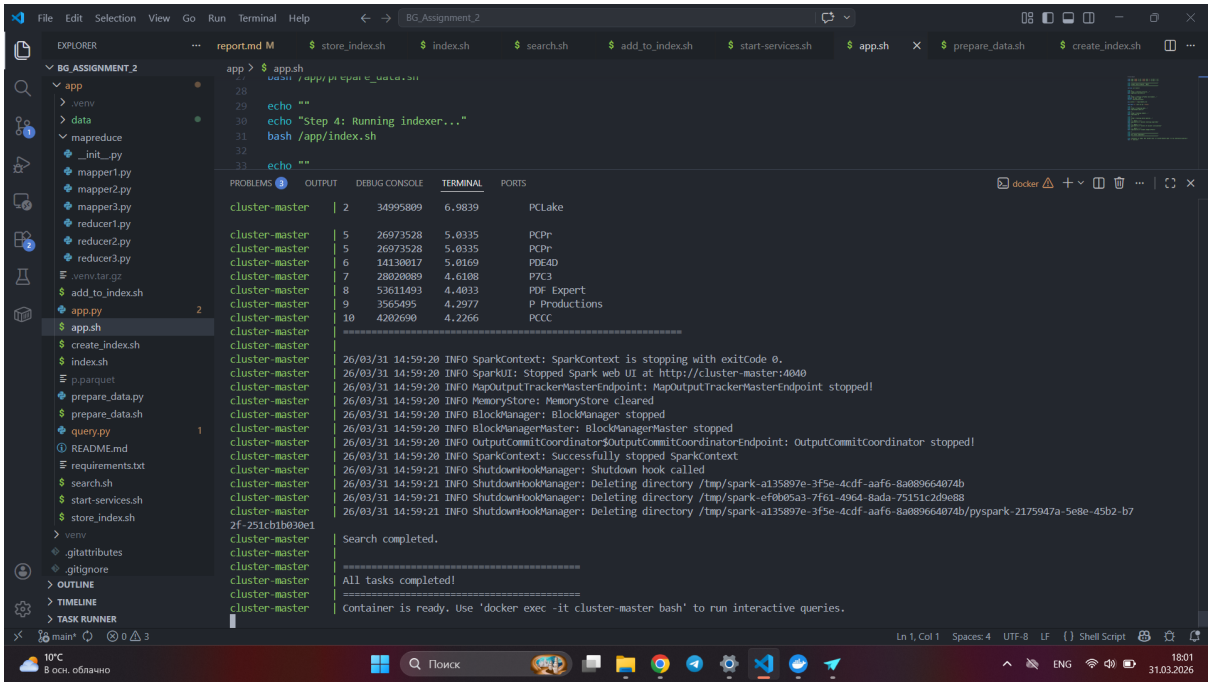

Figure 5:

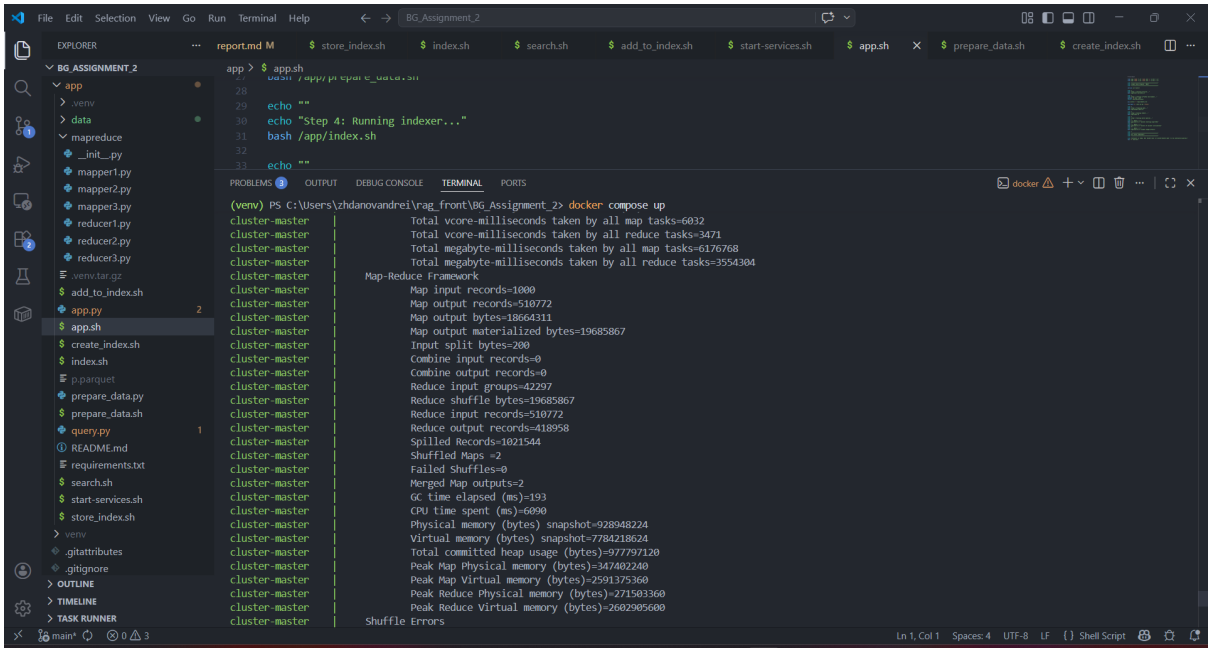

Figure 6:

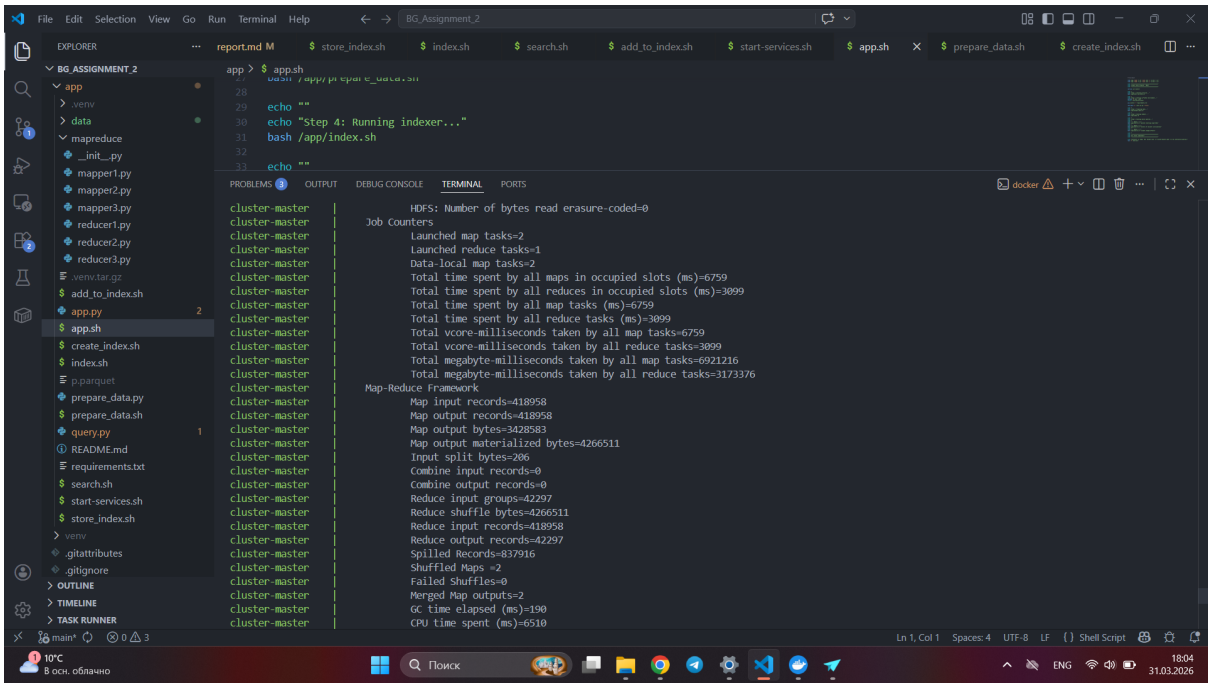

Figure 7:

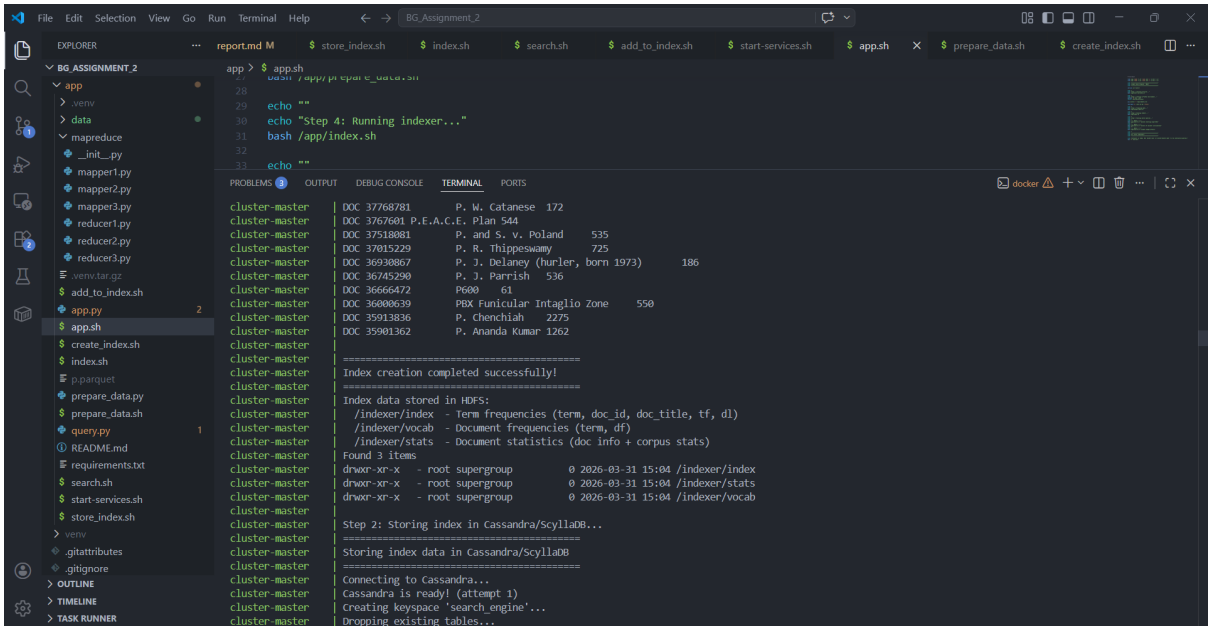

Figure 8:

2.4 Explanation of Retrieved Results and Reflections

2.4.1 Corpus Context

The corpus consists of 1 000 Wikipedia articles whose titles all begin with the letter "P". This domain bias means that queries on general topics (e.g. machine learning, climate change) will rarely find directly relevant documents; the engine can only surface articles that incidentally mention the query terms.

2.4.2 Reflections

Strengths.

- **IDF weighting** effectively suppresses common terms and promotes rare, informative ones.
- **Length normalisation** ($b = 0.75$) prevents long articles from dominating results purely due to higher token counts.
- **Scalable architecture**: MapReduce indexing distributes computation; Cassandra provides fast key-based lookups at query time.

Limitations.

- **No stop-word removal** — words like "the", "is", "of" are still indexed, wasting storage and slightly increasing query latency.
- **No stemming** — "algorithms" does not match "algorithm"; a Porter stemmer would improve recall.
- **Simplified IDF formula**: the implementation uses $\log(N/df)$ instead of the smoothed variant $\log\big((N - df + 0.5)/(df + 0.5)\big)$.

- **Corpus bias**: a "P"-only corpus inherently limits relevance for arbitrary queries — even a perfect ranking algorithm cannot compensate for missing content.

2.5 Adding a Document to the Index (add_to_index.sh)

The `add_to_index.sh` script allows adding a new document to the existing search index. It extracts `doc_id` and `doc_title` from the filename, normalizes the text, uploads the data to HDFS, and triggers a full re-indexing through three MapReduce pipelines followed by storing the results in Cassandra.

2.5.1 Testing

I create test document `99999_Test_Document_Climate_Change.txt` about climate change, permafrost melting in Siberia and the Arctic.

Index state before adding the document:

- Documents: 1000, average length: 541.58, total tokens: 541 576
- Entries in `term_index`: 210 741, in `vocabulary`: 42 297

Command:

```
bash add_to_index.sh data/99999_Test_Document_Climate_Change.txt
```

Index state after adding the document:

- Documents: 1001 (+1), average length: 541.12, total tokens: 541 658 (+82)
- Entries in `term_index`: 210 809 (+68 unique terms), in `vocabulary`: 42 302 (+5 new words)

2.5.2 Search Results

The query `"climate change russia permafrost"` returned the test document at rank 1 with a BM25 score of 29.96, which is 3.9 times higher than the second result (7.76). The high score is due to the presence of rare terms (`permafrost`, `arctic`, `siberia` — df=1) and a high concentration of relevant words in a short document.